Semantic Web

Robin Rausch

1. März 2024

Inhaltsverzeichnis

1 Einführung **1**
1.1 Wissensbank . . . 1
1.2 Inferenzmaschine . . . 1
1.3 Semantische Netze . . . 1
1.4 Frames . . . 1
1.5 Semantische Technologien . . . 2
1.5.1 Syntax und Semantik . . . 2
1.5.2 Weitere Begriffe . . . 2

2 Linked Data, URIs und RDF **2**
2.1 Linked Data . . . 2
2.2 URIs . . . 2
2.3 RDF . . . 2
2.3.1 Datentypen . . . 3
2.3.2 Leere Knoten . . . 5
2.3.3 Mehrwertige Beziehungen . . . 6
2.3.4 Listen . . . 6
2.3.5 RDF Vokabular . . . 7
2.3.6 Zusammenfassung RDF . . . 7
2.4 RDFS . . . 7
2.4.1 rdfs:label und rdfs:comment . . . 8
2.4.2 RDFS Vokabular . . . 9
2.4.3 Ontologien . . . 9
2.5 Semantik . . . 10

3 SPARQL **11**

1 Einführung

1.1 Wissensbank

Eine Wissensbank speichert formalisiertes Wissen. Hat WR-Sprache mit Syntax und Semantik. terminologischen Informationen

1.2 Inferenzmaschine

Eine Inferenzmaschine leitet implizietes Wissen ab. Ist also ein Schlussfolgerungsverfahren.

1.3 Semantische Netze

Bestehen aus Knoten und Kanten. Knoten repräsentieren Individuen, Klassen und Eigenschaften. Kanten repräsentieren is-a und has-prop Beziehungen.
Problematik:

- Mehrdeutigkeit von is-a
- Vererbung als default (keine terminologischen Infos)
- „Semantischer Abstand“ abhängig von syntaktischer Details
- keine formale Semantik
- Netzwerk bedeuted was Ersteller meint
- Konflikte zwischen Benutzer und Programmierer nicht entscheidbar

1.4 Frames

WR-Formalismus zum Beschreiben von Entitäten. Das Ziel war die Imitation von menschlichem Kategorisieren. Logisches Schließen sollte nicht möglich sein. Trennung von repräsntiertem Wissen und Schlussfolgerungsverfahren seien zu radikal.
Syntax:

Class Frame beschreibt Klasse von Entitäten

Instance Frame beschreibt spezifische Entität

slots Frame enthält slots für bestimmte Attribute

filler Wert eines slots wird filler genannt

Defaults für filler sind möglich

Sub-frames für Spezialisierung (is-a)

Schlussfolgerungsverfahren:

Vererbung Sub-frames erben von Default-filler

Kriterialität Wenn Instanz für alle Slots der Class Frame Y einen passenden Filler hat ist diese auch Instanz von Frame Y

Matching Test ob Instanz eines Frames X auch Instanz des Frames Y ist

Probleme von Frames: Frame-Inferenzen lassen sich in Prädikatenlogik ausdrücken und KI als Sammlung von Heuristiken.

1.5 Semantische Technologien

In anderen Worten „Wann wird es Wahr?“.

1.5.1 Syntax und Semantik

$4+=12($ ist syntaktisch falsch.
$3+4=12$ ist syntaktisch richtig, aber semantisch falsch.
$3+4=7$ ist syntaktisch und semantisch richtig.

1.5.2 Weitere Begriffe

Semantische Technologien sind Teilbereich der KI und beschreiben formale Beschreibung in einer Domäne.
Semantic Web ist eine Erweiterung des WWW, die es ermöglicht, Daten mit Bedeutung zu versehen.
Linked Data ist eine Methode, um Daten im Web zu verknüpfen.
Ontologie ist die Lehre vom „Seienden“. Also unveränderliche und unvergängliche Ideen. Der Gegenbegriff wäre „Werdendes“. In der Informatik ist die Ontologie repräsentiertes Wissen. Beispielsweise Inhalt einer Wissensbank oder Instanzen von Klassen. Ontologie besser beschreiben/definieren
Schlagwort-Verfahren bei beispielsweise Browsern kann funktionieren, muss aber nicht (Beispiel: Gorillas und Dunkelhäutige Menschen von Google KI). W3C hat Semantic Web Standards entwickelt.

2 Linked Data, URIs und RDF

2.1 Linked Data

URIs sind Namen für Dinge.
Verwende HTTP URIs, damit Menschen und Maschinen darauf zugreifen können.
Wenn jemand eine URI nachschaut, sollte er nützliche Informationen erhalten die einen Standard verwenden.
Verwende dort weitere URIs sodass Menschen weitere Informationen finden können.

2.2 URIs

Uniform Resource Identifier. Zeichenfolge, die eine abstrakte oder physische Ressource identifiziert.
URI ist ein Link, URL ist Teilmenge von URI und L steht für Locator. URN identifiziert Dinge (Menschen, Bücher, ...). URL liefert gesuchtes Objekt, URN liefert Datei die gesuchte Objekt beschreibt.

2.3 RDF

Resource Description Framework. Wird verwendet um Aussagen über Ressourcen zu machen. Besteht aus URIs, Literale und Leeren Knoten. URIs sind Namen für Ressourcen. Literale sind Werte. Leere Knoten sind anonyme Ressourcen.

RDF-Graph ist eine Menge von Tripeln. Tripel besteht aus Subjekt, Prädikat und Objekt.

Subjekt URI oder leerer Knoten

Prädikat URI

Objekt URI, Literal oder leerer Knoten

Als Syntax für RDF gibts XML, Notation 3, N-Triples und Terse Triple Language. Wir benutzen ausschließlich Turse Triple Language(TTL, bzw. „Trutle“).

Beispiel:

Beispiel 2.2 (Tripel in TTL-Notation)

```
<http://example.org/SemanticWeb> <http://example.org/verlegtBei>
   <http://springer.com/Verlag> .
<http://example.org/SemanticWeb> <http://example.org/titel>
   "Semantic Web - Grundlagen" .
<http://springer.com/Verlag> <http://example.org/name> "Springer Verlag" .
```

Beispiel in abgekürzter Version:

Beispiel 2.3 (Abkürzungen in TTL)

```
@prefix ex: <http://example.org/> .
@prefix springer: <http://springer.com/> .
@prefix : <http://jan.org/ontology/> .

ex:SemanticWeb ex:verlegtBei springer:Verlag .
ex:SemanticWeb ex:titel "Semantic Web - Grundlagen" .
springer:Verlag ex:name "Springer Verlag" .
:jan :vorname "Jan".
```

Weitere Beispiele:

Mehrere Tripel mit gleichem Subjekt kann man zusammenfassen:

Beispiel 2.4

```
@prefix ex: <http://example.org/> .
@prefix springer: <http://springer.com/> .

 ex:SemanticWeb    ex:verlegtBei   springer:Verlag ;
                   ex:titel        "Semantic Web - Grundlagen" .
 springer:Verlag   ex:name         "Springer Verlag" .
```

Ebenso Tripel mit gleichem Subjekt und Prädikat:

Beispiel 2.5

```
@prefix ex: <http://example.org/> .

 ex:SemanticWeb   ex:autor   ex:Hitzler, ex:Krötzsch,
                             ex:Rudolph, ex:Sure ;
                  ex:titel   "Semantic Web - Grundlagen" .
```

2.3.1 Datentypen

Standardmäßig sind alles Literale („100“ < „11“) ähnlich wie Strings. Datentypen können aber auch explizit angegeben werden werden.

Datentyp

- Gruppe von gleichartigen Literalen
- referenziert durch URI ⇝ neue Datentypen möglich
- Präfix für XML-Schema-Datatypen: `xsd: <http://www.w3.org/2001/XMLSchema#>`
- verpflichtend
  - `xsd:string` Zeichenketten (default für untypisierte Literale)
  - `rdf:langString` Zeichenketten mit Sprachangabe
- standardisiert
  - `xsd:integer` Ganzzahlen
  - `xsd:decimal` Dezimalzahlen
  - `xsd:boolean` {true, false}
  - `xsd:date` Datum im Format `jjjj-mm-tt`
  - `xsd:time` Zeit im Format `hh:mm:ss.sss...`

Um den Datentyp angzugeben, muss dieser Syntax eingehalten werden:

```
ex:name "Springer Verlag"^^xsd:string ;
```

Sprachinformationen werden folgendermaßen angegeben:

```
<http://springer.com/Verlag> ex:name "Springer Verlag"@de ,
                                     "Springer Science+Business Media"@en .
```

Der Datentyp hier lautet rdf:langString.

2.3.2 Leere Knoten

URI repräsentiert abstrakte oder physische Ressource
Literal repräsentiert sich selbst
Leerer Knoten repräsentiert Ressource, die nicht benannt werden kann oder soll

- Englisch: bnode („blank node“)
- hat keinen URI, sondern Identifier (ID) mit Präfix „_:“
- kann als Existenzaussage gelesen werden

Beispiel 2.9 („John liebt irgendjemanden.“)

```
ex:John ex:liebt _:id1 .
```

Beispiel 2.10 („Irgendjemand liebt Mary.“)

```
_:id1 ex:liebt ex:Mary .
```

Vorsicht: „John steht in irgendeiner Beziehung zu Mary.“

```
ex:John _:id1 ex:Mary .
```

Bnodes sind als Prädikate unzulässig!

- mit Präfix „_:“

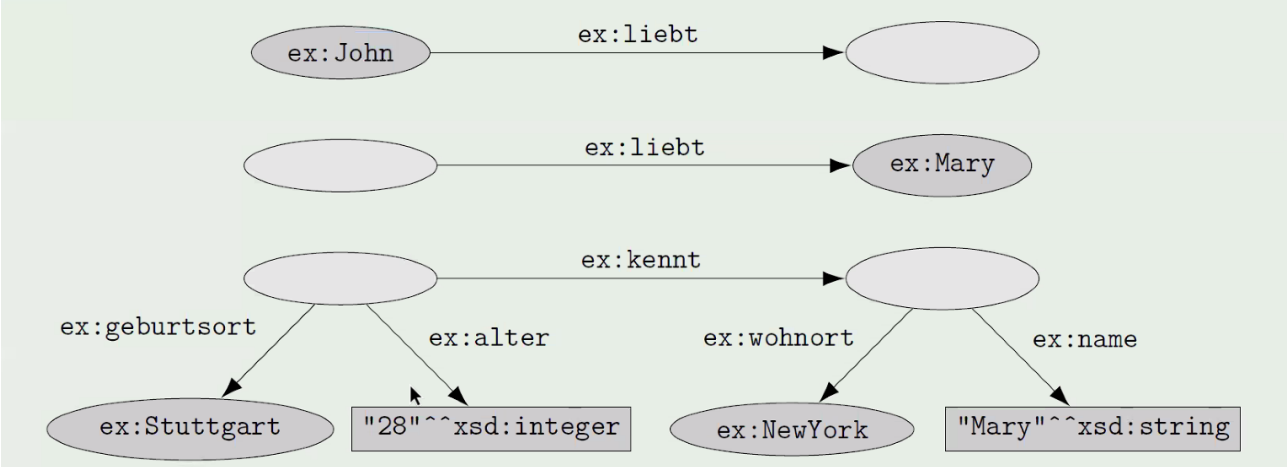
Beispiel 2.11 („Jemand, der 28 Jahre alt ist und in Stuttgart geboren ist, kennt jemanden, der Mary heißt und in New York wohnt.“)

```
_:id1 ex:alter "28"^^xsd:integer ;
      ex:geburtsort ex:Stuttgart ;
      ex:kennt _:id2 .
_:id2 ex:name "Mary"^^xsd:string ;
      ex:wohnort ex:NewYork .
```

- mit eckigen Klammern [ ]
  - vermeidet Notwendigkeit, einen Namen zu vergeben
  - als Subjekt oder Objekt möglich
  - zusätzliche Tripel mit Bnode als Subjekt können zwischen Klammern angegeben werden

Beispiel 2.12

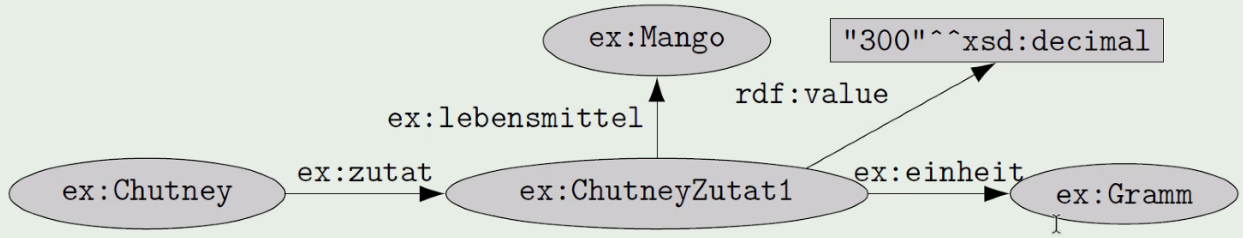
```
[ ex:alter "28"^^xsd:integer ;
  ex:geburtsort ex:Stuttgart ] ex:kennt [ ex:name "Mary"^^xsd:string ;
                                          ex:wohnort ex:NewYork ] .
```



2.3.3 Mehrwertige Beziehungen

Mehrwertige Beziehungen wie in einem Rezept beispielsweise (Liste an Zutaten mit Mengenangaben) sind in RDF nicht möglich. Hierzu müssen Hilfsknoten verwendet werden:

- Als Graph



- in TTL-Syntax

```
ex:Chutney        ex:zutat ex:ChutneyZutat1 .
ex:ChutneyZutat1  ex:lebensmittel ex:Mango ;
                  rdf:value "300"^^xsd:decimal ;
                  ex:einheit ex:Gramm .
```

Da ChutneyZutat1 ein blöder Begriff ist, kann das durch BNodes (Leere Knoten) ersetzt werden:

Beispiel 2.22 (TTL-Syntax)

```
ex:Chutney ex:zutat _:id1 .
_:id1 ex:lebensmittel ex:Mango ;
      rdf:value "300"^^xsd:decimal ;
      ex:einheit ex:Gramm .
```

Beispiel 2.23 (Verkürzte TTL-Schreibweise)

```
ex:Chutney ex:zutat
    [ ex:lebensmittel ex:Mango ;
      rdf:value "300"^^xsd:decimal ;
      ex:einheit ex:Gramm ] ,
    [ ex:lebensmittel ex:Cayennepfeffer ;
      rdf:value "1"^^xsd:decimal ;
      ex:einheit ex:Teeloeffel ] .
```

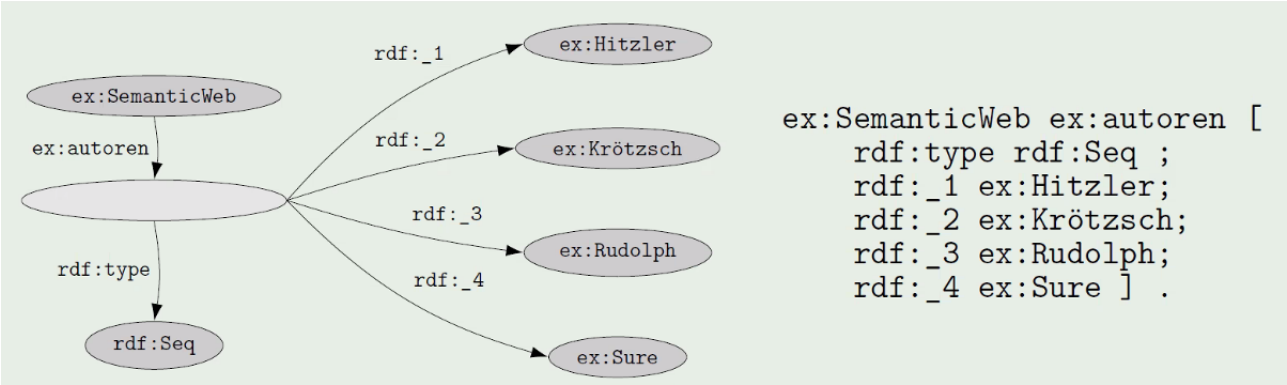
2.3.4 Listen

Für Listen in RDF gibt es zwei Varianten:

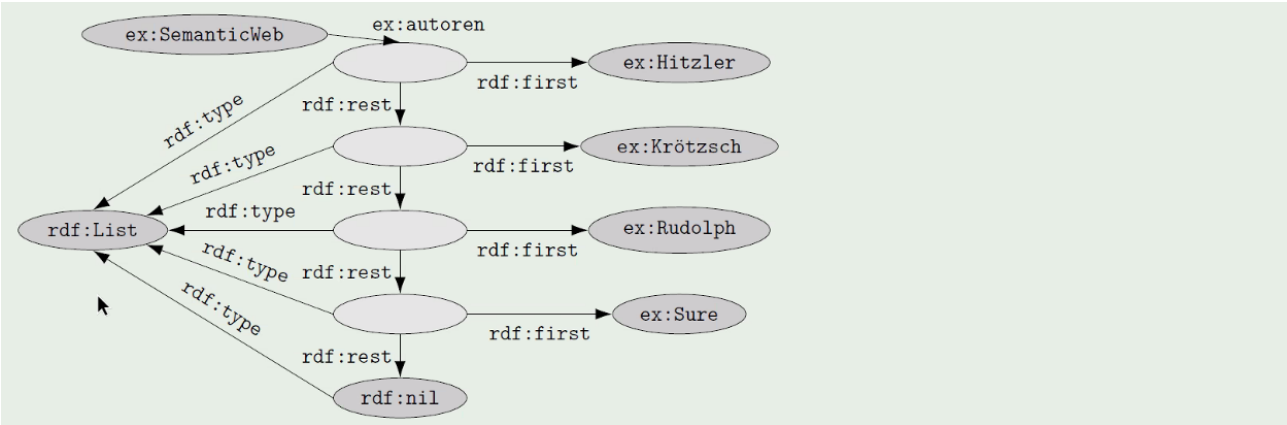
Container offene Liste
- Hinzufügen von neuen Einträgen möglich
- Typ wird Wurzelknoten via `rdf:type` zugewiesen
  - `rdf:Seq` geordnete Liste (Folge)
  - `rdf:Bag` ungeordnete Liste (Menge)
  - `rdf:Alt` liste alternativer Möglichkeiten zur Auswahl

Collections geschlossene Liste
- Hinzufügen von neuen Einträgen nicht möglich

Beispiel für Listentyp Container:



Iterieren durch eine Liste:



2.3.5 RDF Vokabular

- Klassen

`rdf:Property`	Klasse aller Properties (Ressourcen, die als Prädikate fungieren können)
`rdf:Seq`, `rdf:Bag`, `rdf:Alt`	Klassen verschiedener Arten von Containern
`rdf:List`	Klasse aller Collections

- Properties

`rdf:type`	Instanz-Beziehung
`rdf:value`	Zahlenwerte
`rdf:li`, `rdf:_1`, `rdf:_2`, ...	für Container
`rdf:first`, `rdf:rest`	für Collections

- Sonstige Ressourcen

`rdf:nil`	leere Collection

2.3.6 Zusammenfassung RDF

RDF ist universeller Standard für Speicherung und Austausch von Daten. Graphen-basiert und leicht zu verstehen. Unterstützt global eindeutige Variablen.
Die Grenzen von RDF sind, dass der Schwerpunkt auf Individuen liegt (Faktenwissen), kaum Möglichkeiten zur Kodierung von Schemawissen exsitiert und dass RDF eine intuitive Semantik hat (Bedeutungen von Vokabularen müssen Autoren im Kopf bleiben).

2.4 RDFS

Grund für RDFS ist, dass man Aussagen zu Klassen anstatt nur Individuen machen will (Schemawissen).

Beispiel 2.29 (Jedes Lehrbuch ist ein Buch)

`ex:SemanticWeb rdf:type ex:Lehrbuch .`

Problem Suche nach Instanzen von `ex:Buch` liefert kein Resultat

Ansatz 1 `ex:SemanticWeb rdf:type ex:Buch .`

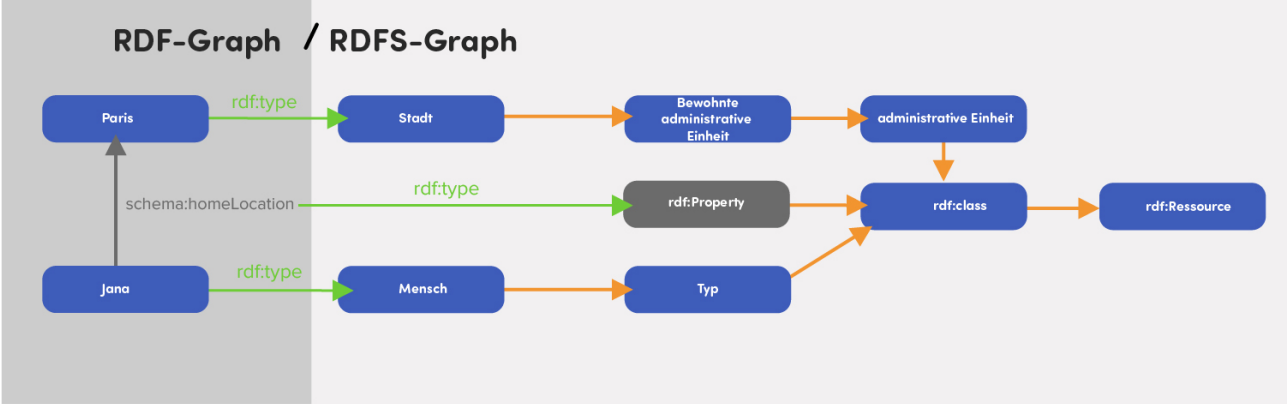
- Löst das Problem nur für eine Ressource `ex:SemanticWeb`
- Hinzufügen für alle Instanzen führt zu unnötig großen RDF-Graphen

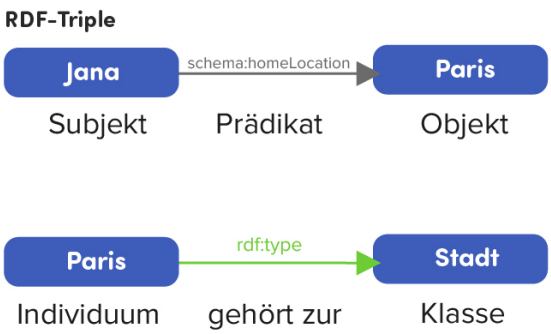
Ansatz 2 Allgemeine Aussage, dass jedes Lehrbuch auch ein Buch ist

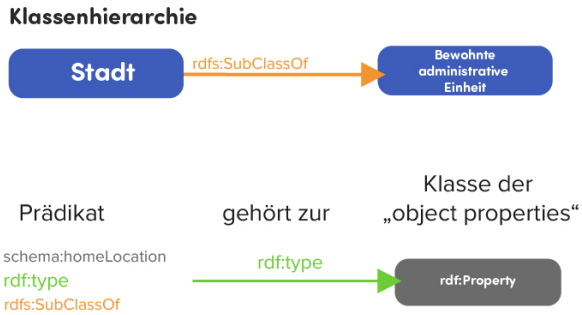
- jede Instanz von `ex:Lehrbuch` ist automatisch auch Instanz von `ex:Buch`
- Property `rdfs:subClassOf`

`ex:Lehrbuch rdfs:subClassOf ex:Buch .`

„Die Klasse der Lehrbücher ist eine Unterklasse der Klasse der Bücher.“







rdf:type kann durch „a“ angekürzt werden.

2.4.1 rdfs:label und rdfs:comment

Werden zum Hinzufügen von Anmerkungen verwendet und verbessern somit das Verständnis.

2.4.2 RDFS Vokabular

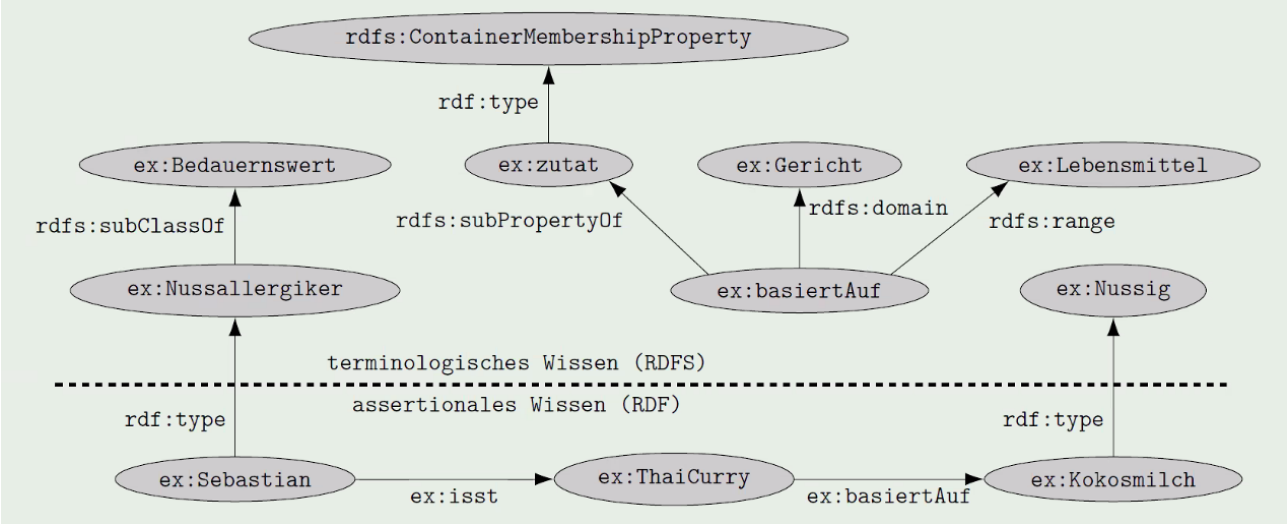
- Klassen

  `rdfs:Resource` Klasse aller Ressourcen (sämtliche Elemente der Domäne)
  `rdfs:Class` Klasse aller Klassen
  `rdfs:Container` Oberklasse von `rdf:Seq`, `rdf:Bag` und `rdf:Alt`
  `rdfs:ContainerMembershipProperty` Klasse aller Properties, die eine Enthaltenseinsbeziehung darstellen, z. B. Container, Collections
  `rdfs:Literal` Klasse aller Literalwerte (Oberklasse aller Datentypen)
  `rdfs:Datatype` Klasse aller Datentypen

- Properties

  `rdfs:subClassOf` Unterklassenbeziehung
  `rdfs:subPropertyOf` Unterpropertybeziehung
  `rdfs:domain` Definitionsbereich
  `rdfs:range` Zielbereich
  `rdfs:member` Oberproperty aller ContainerMembershipProperties
  `rdfs:label`, `rdfs:comment`, `rdfs:isDefinedBy`, `rdfs:seeAlso` Annotationen

2.4.3 Ontologien



2.5 Semantik

Definition 2.38 (Einfache Interpretation)

Ein Vokabular $V = (S, L)$ besteht aus einer Menge S von URIs und einer Menge L von Literalen.
Eine einfache Interpretation $\mathcal{I}$ für V besteht aus

- IR, einer nichtleeren Menge von Ressourcen,
- IP, einer Menge von Properties,
- $I_S : S \to IR \cup IP$, einer Funktion von den URIs aus V in die Vereinigung von IR und IP,
- $I_L : L \to IR$, einer Funktion von den Literalen aus V nach IR, und
- $I_{EXT} : IP \to 2^{IR \times IR}$, einer Funktion, die jeder Property aus IP eine Menge von Paaren aus IR zuordnet.

- IR wird Domäne oder Universum von $\mathcal{I}$ genannt
- $I_{EXT}(p)$ wird Extension der Property p genannt

Definition 2.39 (Datentyp)

Ein RDF-Datentyp D besteht aus

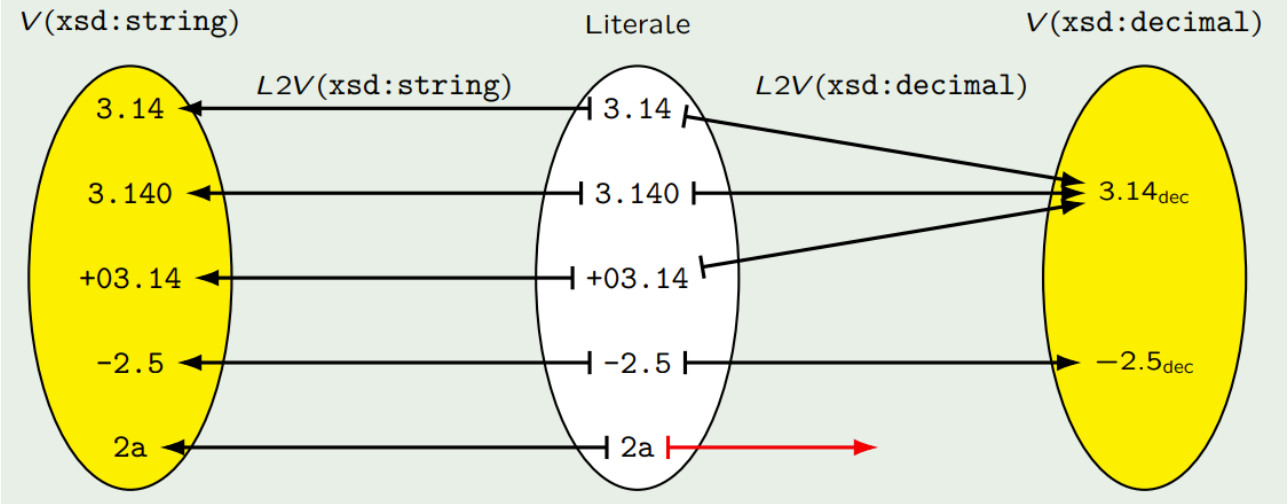
- lexikalischer Bereich $L(D)$: Menge der zulässigen Strings
- Wertebereich $V(D)$: Zielmenge
- Abbildung $L2V(D)$ von L auf V

URIs I_S kann URIs auf beliebige Elemente von $IR \cup IP$ abbilden
Literale für I_L ist die Interpretationsfunktion durch den Datentyp vorgegeben

- $I_L(\mathtt{l\hat{}\hat{}d}) = (L2V(\mathtt{d}))(\mathtt{l})$
- $I_L(\mathtt{l\hat{}\hat{}d})$ ist undefiniert, wenn $\mathtt{l} \notin L(\mathtt{d})$ gilt

Beispiel 2.40 (Datentyp xsd:integer)

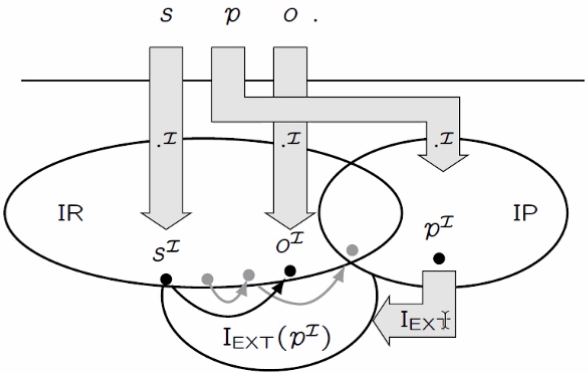
- $L(\mathtt{xsd:integer}) = (+|-|\varepsilon) \cdot [0..9]^+$
- $V(\mathtt{xsd:integer}) = \mathbb{Z}$
- $L2V(\mathtt{xsd:integer}) = \{0 \mapsto 0_{dec}; 01 \mapsto 1_{dec}; -0050 \mapsto -50_{dec}, \ldots\}$



Definition 2.44 (Modell)

Eine Interpretation $\mathcal{I}$ ist Modell des Tripels $s\ p\ o\ .$, wenn

- s, p und o in V enthalten sind und
- $(s^{\mathcal{I}}, o^{\mathcal{I}}) \in \mathrm{I_{EXT}}(p^{\mathcal{I}})$ gilt.



3 SPARQL

sample